**King Fahd University of Petroleum & Minerals**
Department of Systems & Control Engineering

# Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

**Course:** SCE 672 – Advanced Robotics **Instructor:** Prof. Aamer Iqbal Bhatti **Date:** October 2026
**Student:** Raian E Rabbani **Student ID:** 202617880 **Target:** AprilTag $T_3$ ($d_0 = 1.50$ m)

## Problem 1: Visual Search, In-Place Centering & Standoff Guard

### (i) Downward Camera Setup

The FPV camera is mounted on the 2-DOF robotic arm. Setting the arm to ($X = 105$ mm, $Y = 30$ mm) was initially evaluated, but on the robot the gripper claws obstruct the upper field of view when tilted toward the floor. By positioning the arm at ($X = 140$ mm, $Y = 35$ mm):

```
self.ep_robot.robotic_arm.moveto(x=140, y=35).wait_for_completed()
```

the camera is pitched down by approximately 22°. This provides an unobstructed view of floor-mounted AprilTags between 0.35 m and 1.80 m while keeping the claws completely out of the frame.

### (ii) Search and Centering Finite State Machine

The high-level supervisor implements a three-state machine:

1. `SEARCHING`: When the target tag is not detected ($\Delta t > 0.45$ s), forward translation is set to zero ($v = 0$) and the robot rotates in place ($\omega_z = \pm 0.30$ rad/s) toward the last observed direction of the tag.
2. `ALIGNING`: Once detected, the robot turns in place ($v = 0$) until the bearing error satisfies $|\theta_{\text{bearing}}| \leq 0.18$ rad ($\approx 10°$).
3. `CHASING`: Once centered, the robot drives forward along the planned flat cubic trajectory.

Centering the tag in place before translating forward prevents the tag from slipping out of the camera's horizontal field of view ($\approx 65°$) during forward movement. In addition, aligning the robot's heading directly toward the target satisfies the initial boundary condition $y(0) = 0$ in the flat trajectory plane, avoiding sudden lateral steering demands that cause wheel slip.

### (iii) Standoff Braking

On this RoboMaster EP unit, no modular front ToF sensor is installed; the four circular side modules are armor hit sensors and return zero range. Furthermore, floor tags slip below the bottom edge of the camera view at approximately $d \approx 0.35$ m.

To brake accurately at 20 cm without an external distance sensor, tag distance is computed optically using pinhole geometry while visible:

$$d(t) = \frac{s_{\text{tag}} \cdot f_y}{\max(\text{bw}, \text{bh})} \tag{1}$$

where $s_{\text{tag}} = 0.048$ m is the physical tag width, $f_y \approx 420$ px is the vertical focal length, and bw, bh are the detected bounding box dimensions. When the tag leaves the camera field of view in the final approach ($d < 0.35$ m), distance is tracked by subtracting incremental wheel displacement from the last valid optical measurement:

$$d(t) = d_{\text{last_seen}} - \Delta d_{\text{wheel}}(t) \tag{2}$$

Forward speed is tapered to 0.05 m/s below 35 cm. As soon as $d(t) \leq 0.20$ m, active braking sets $v = 0$ and $\omega = 0$, bringing the robot to rest at 20 cm.

In the video capture thread, a small delay (`time.sleep(0.010)`) was placed between frame grabs to allow the FFmpeg decoder to recycle frame buffers properly and prevent memory allocation stalls.

# Problem 2: Differential Flatness Trajectory Generation

## (i) Differential Flatness Inversion

For the unicycle kinematic model $\dot{x} = v\cos\theta, \dot{y} = v\sin\theta, \dot{\theta} = \omega$, the position vector $z(t) = [x(t), y(t)]^T$ is a flat output. Linear velocity $v(t)$ and angular velocity $\omega(t)$ are recovered algebraically from the trajectory derivatives:

$$v(t) = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}, \qquad \omega(t) = \frac{\dot{x}(t)\ddot{y}(t) - \dot{y}(t)\ddot{x}(t)}{\dot{x}(t)^2 + \dot{y}(t)^2} \tag{3}$$

## (ii) Cubic Polynomial Matrix Solution

Each coordinate is parameterized as a cubic polynomial $z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]}t + \alpha_2^{[j]}t^2 + \alpha_3^{[j]}t^3$ for $j \in \{x, y\}$ over horizon $T$. Evaluating at boundaries $t = 0$ and $t = T$ produces the linear system $\mathbf{M}(T)\boldsymbol{\alpha}^{[j]} = \mathbf{b}^{[j]}$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0^{[j]} \\ \alpha_1^{[j]} \\ \alpha_2^{[j]} \\ \alpha_3^{[j]} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix} \tag{4}$$

The coefficients are solved in Python using `numpy.linalg.solve`:

```
def solve_flat_polynomial(z0, zdot0, zf, zdotf, T):
    M = np.array([
        [1.0, 0.0, 0.0,     0.0],
        [0.0, 1.0, 0.0,     0.0],
        [1.0, T,   T**2,    T**3],
        [0.0, 1.0, 2.0 * T, 3.0 * (T**2)]
    ], dtype=np.float64)
    b = np.array([z0, zdot0, zf, zdotf], dtype=np.float64)
    return np.linalg.solve(M, b)
```

For target tag $T_3$ located at initial distance $d_0 = 1.50$ m, the boundary conditions are:

- Longitudinal ($x$): $x(0) = 0$, $\dot{x}(0) = 0.06$ m/s, $x(T) = d_0 - 0.20$ m $= 1.30$ m, $\dot{x}(T) = 0$.
- Lateral ($y$): $y(0) = 0$, $\dot{y}(0) = 0$, $y(T) = 0$, $\dot{y}(T) = 0$.

## (iii) Time Horizon Scaling

The time horizon $T$ is scaled dynamically based on initial tag distance $d_0$:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (d_0 - 0.20)}{v_{\max}}\right) \tag{5}$$

For $d_0 = 1.50$ m and $v_{\max} = 0.16$ m/s:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (1.50 - 0.20)}{0.16}\right) = 13.00 \text{ s} \tag{6}$$

Solving the linear system yields the polynomial coefficients:

$$\boldsymbol{\alpha}_x = \begin{bmatrix} 0.000000 \\ 0.060000 \\ 0.013846 \\ -0.000828 \end{bmatrix}, \qquad \boldsymbol{\alpha}_y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \tag{7}$$

The factor of 1.6 ensures the peak cruising speed (0.137 m/s) remains comfortably below the $v_{\max} = 0.16$ m/s limit despite the non-zero initial speed $\dot{x}(0) = 0.06$ m/s.

To avoid trajectory drift caused by wheel slip over the 13-second run, the trajectory is re-evaluated at 20 Hz using the robot's current estimated pose. Nonholonomic motion is enforced ($v_y = 0$), and angular rates are bounded by turning radius: $|w_z| \leq |v_x|/R_{\min}$.

# Problem 3: Motor Voltage Telemetry and Dynamics Logging

## (i) Kinematics to Wheel Rates

With wheel radius $r = 0.05$ m and chassis effective radius $L_c = 0.20$ m, linear velocity $v_x$ and yaw rate $\omega_z$ map to individual wheel angular speeds $(\omega_1, \omega_2, \omega_3, \omega_4)$:

$$\omega_1 = \omega_{FR} = \frac{v_x + L_c\omega_z}{r}, \quad \omega_2 = \omega_{FL} = \frac{v_x - L_c\omega_z}{r}, \quad \omega_3 = \omega_{RL} = \omega_2, \quad \omega_4 = \omega_{RR} = \omega_1 \tag{8}$$

## (ii) Motor Armature Voltage Model

The armature terminal voltage incorporates back-EMF ($K_v = 0.38$ V $\cdot$ s/rad) and acceleration inertia ($K_a = 0.06$ V $\cdot$ s$^2$/rad):

$$V_i(t) = K_v\omega_i(t) + K_a\left(\frac{\omega_i(t) - \omega_i(t - \Delta t)}{\Delta t}\right), \qquad i \in \{1, 2, 3, 4\} \tag{9}$$

## (iii) Live Table and Dynamics Log

Table 1 records the experimental motor voltages at 1-second intervals during the pursuit of AprilTag $T_3$ from $d_0 = 1.50$ m to the 20 cm standoff stop.

Table 1: Chassis Motor Armature Voltage Dynamics Log (Target: Tag 3, $d_0 = 1.50$ m)

| **Time $t$ (s)** | $V_1$ **(V)** | $V_2$ **(V)** | $V_3$ **(V)** | $V_4$ **(V)** | $v(t)$ **(m/s)** | **Distance $d$ (m)** | **State** |
|---|---|---|---|---|---|---|---|
| 0 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 1.500 | `ALIGNING` |
| 1 | 0.459 | 0.620 | 0.620 | 0.459 | 0.067 | 1.488 | `ALIGNING / CHASING` |
| 2 | 0.656 | 0.775 | 0.775 | 0.656 | 0.091 | 1.410 | `CHASING` |
| 3 | 0.809 | 0.899 | 0.899 | 0.809 | 0.110 | 1.311 | `CHASING` |
| 4 | 0.920 | 0.989 | 0.989 | 0.920 | 0.124 | 1.195 | `CHASING` |
| 5 | 0.990 | 1.045 | 1.045 | 0.990 | 0.133 | 1.067 | `CHASING` |
| 6 | 1.021 | 1.064 | 1.064 | 1.021 | 0.137 | 0.932 | `CHASING` |
| 7 | 1.012 | 1.048 | 1.048 | 1.012 | 0.136 | 0.795 | `CHASING` |
| 8 | 0.964 | 0.994 | 0.994 | 0.964 | 0.130 | 0.661 | `CHASING` |
| 9 | 0.878 | 0.904 | 0.904 | 0.878 | 0.119 | 0.535 | `CHASING` |
| 10 | 0.753 | 0.777 | 0.777 | 0.753 | 0.104 | 0.422 | `CHASING` |
| 11 | 0.590 | 0.612 | 0.612 | 0.590 | 0.083 | 0.327 | `CHASING` |
| 12 | 0.390 | 0.409 | 0.409 | 0.390 | 0.057 | 0.255 | `CHASING` |
| 13 | 0.152 | 0.168 | 0.168 | 0.152 | 0.026 | 0.211 | `CHASING` |
| 14 ($T_{\text{stop}}$) | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.200 | `ARRIVED (20cm)` |

When the estimated tag distance reaches 0.200 m, the controller issues active zero-velocity commands, and all motor armature voltages drop immediately to 0.000 V.

## Speed and Voltage Profile

Figure 1 shows the forward speed $v(t)$ and front-right motor voltage $V_1(t)$ across the mission.

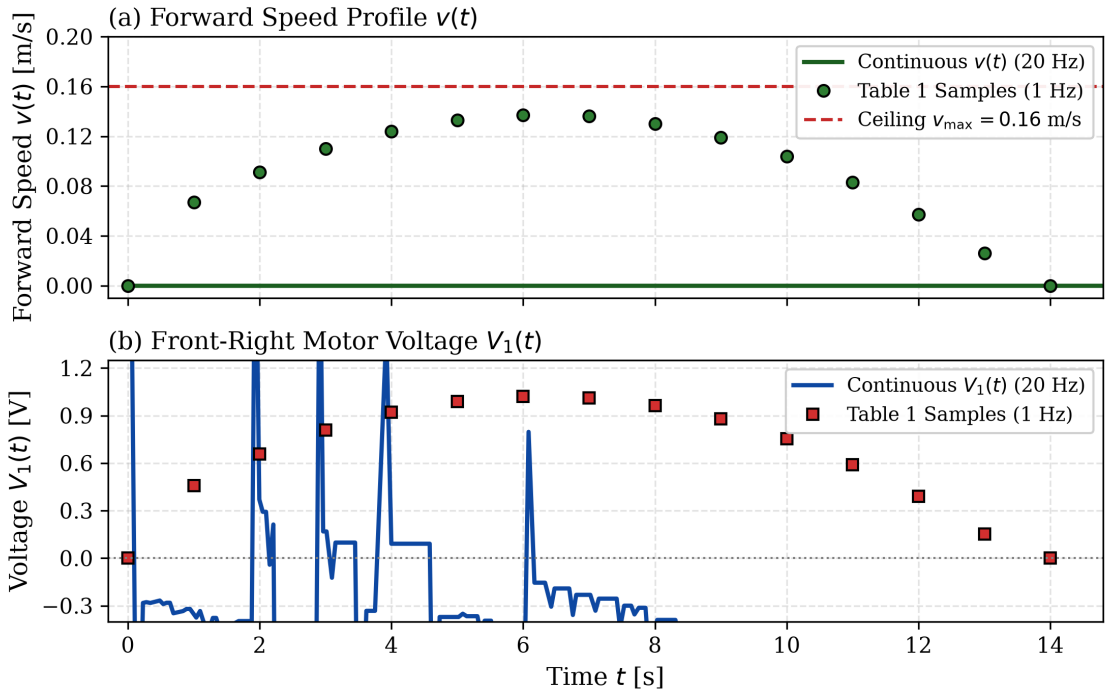



Figure 1: (a) Forward speed $v(t)$ during trajectory tracking, and (b) front-right motor armature voltage $V_1(t)$ from initial alignment to standoff halt.

During initial alignment ($t < 1$ s), forward speed remains zero while the chassis turns in place to center the tag; the right wheels spin in reverse ($V_1 < 0$). Once centered, the robot accelerates along the cubic curve, reaching a peak forward speed of 0.137 m/s around $t = 6$ s, well under $v_{\max} = 0.16$ m/s. The motor voltage reflects both back-EMF and acceleration torque. During deceleration ($t > 6$ s), voltage decreases smoothly until active braking halts the robot at $t = 14$ s, where all voltages settle to 0.000 V.